

High Performance Computing

Pacheco, Chapter 1

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

Copyright © 2013—2025

Moreno Marzolla, Università di Bologna, Italy

<https://www.moreno.marzolla.name/teaching/HPC/>



Except where stated otherwise, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). To view a copy of this license, visit <https://creativecommons.org/licenses/by-sa/4.0/> or send a letter to Creative Commons, 543 Howard Street, 5th Floor, San Francisco, California, 94105, USA.

Credits

- prof. Salvatore Orlando, Univ. Ca' Foscari di Venezia
<http://www.dsi.unive.it/~orlando/>
- prof. Mary Hall, University of Utah
<https://www.cs.utah.edu/~mhall/>
- Tim Mattson, Intel
- Charles Leiserson, *Performance Engineering of Software Systems*, MIT OpenCourseware

Who I am

- Moreno Marzolla
 - moreno.marzolla@unibo.it
 - <https://www.moreno.marzolla.name/>
- Teaching
 - High Performance Computing (LT ISI)
 - Calcolo Parallelo (LM INF)
 - Architectures and Platforms for AI (LM AI)
 - Past: Algorithms and Data Structures; Complex Systems; Software Engineering; Programming
- Research
 - Parallel and distributed algorithms
 - Modeling and simulation

High Performance Computing

- Web page
 - <https://www.moreno.marzolla.name/teaching/HPC/>
- Schedule
 - Wednesday 9:00–12:00 room 2.13
 - Thursday 15:00–17:00 lab 2.2
- Lectures will be recorded
- Office hours
 - Wednesday, after the lecture
 - Online (Teams); please [send e-mail](#)

What you will learn

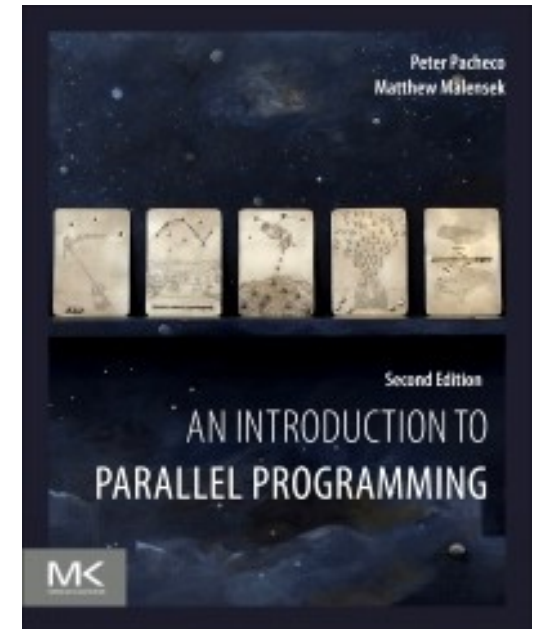
- What parallel computers are, and how they work
- How to design parallel programs
- How to program parallel computers
- How to evaluate the efficiency of parallel programs

Syllabus

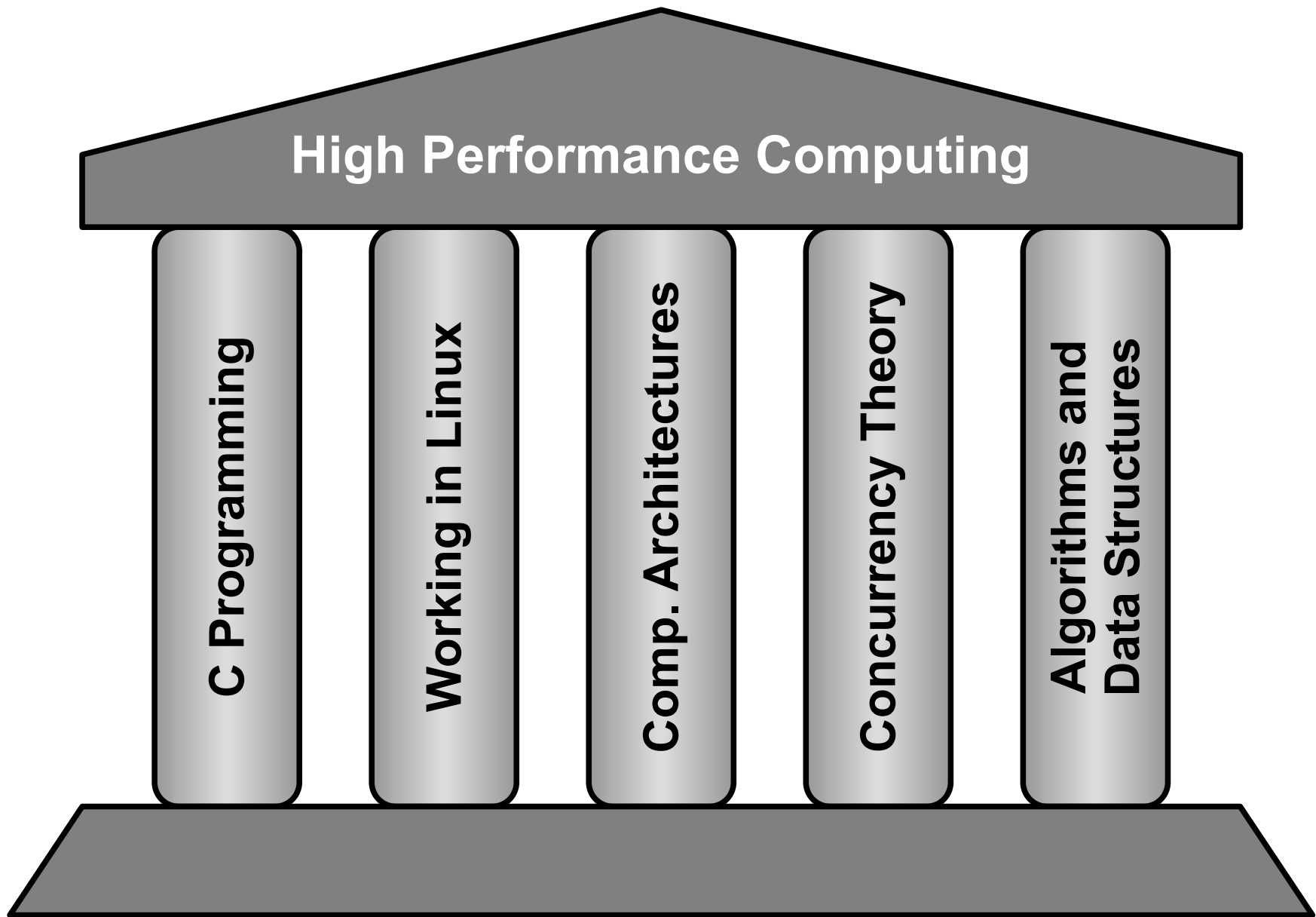
- 6 CFU
 - ~4 CFU lectures
 - ~2 CFU lab sessions
- Theory (first ~3 weeks)
 - Parallel architectures
 - Parallel programming patterns
 - Performance evaluation of parallel programs
- Programming parallel architectures (rest of the course)
 - **Shared-memory** programming with OpenMP
 - **Distributed-memory** programming with MPI
 - **GPU** programming with CUDA
 - (time permitting) **SIMD** programming

Textbook

- Peter Pacheco, Matthew Malensek,
An Introduction to Parallel Programming, 2nd ed., Morgan Kaufmann, 2021, ISBN 9780128046050
 - The 1st edition is ok, too
- *CUDA C programming guide*
<http://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- Slides and other teaching material
<https://www.moreno.marzolla.name/teaching/HPC/>



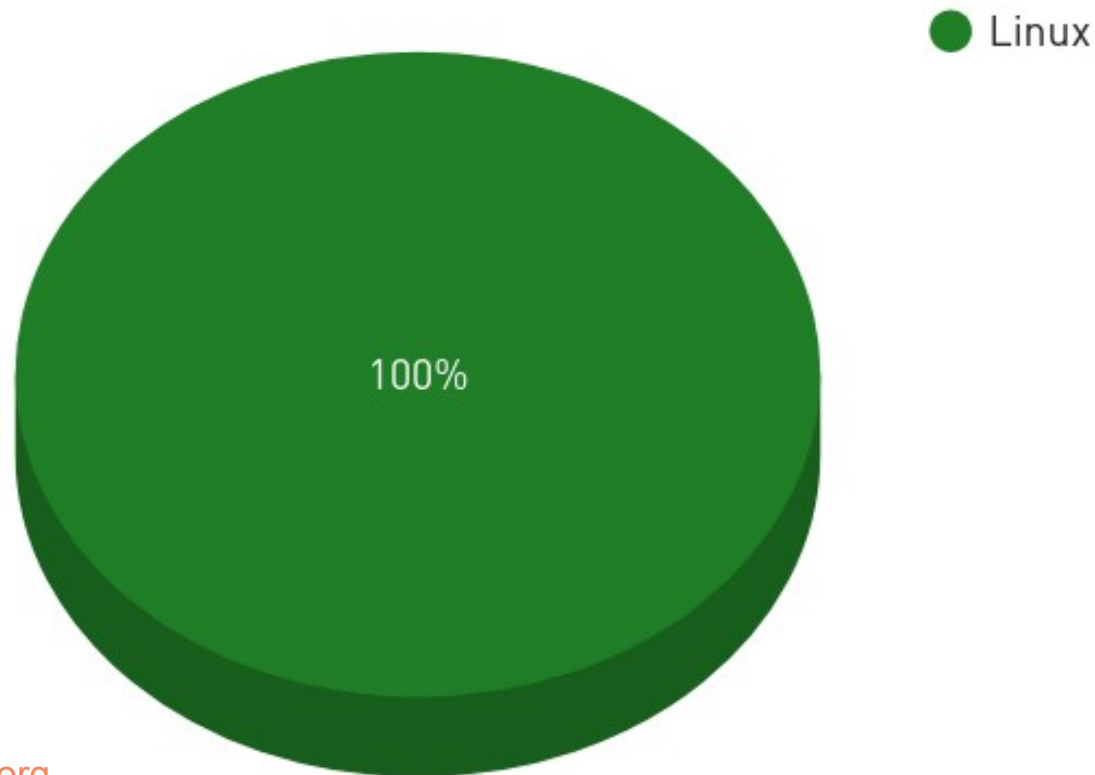
Prerequisites



Lab sessions

- Hands-on programming exercises
- We will work under **Linux**

Operating system Family System Share



Exam

- **Written exam** (weight: 40%)
 - In-person, closed book, using EOL
 - Open questions and/or simple exercises on all topics discussed during the course
 - 6 dates: 3 during jan/feb 2026; 2 during jun/jul 2026; 1 in the fall term sep 2026
 - You can refuse the grade and retry the written exam
- **Individual programming project + written report** (weight: 60%)
 - Specification defined by the instructor; the same project for the whole academic year
 - Project can be handed in **at any time before September 30, 2026**
 - There is no discussion, unless I request it
 - **If you refuse the grade, you must hand-in a NEW project on NEW specifications**
- Final grade rounded to the nearest integer
- The written exam and programming project are **independent**, and **can be done in any order**
- This is the **last year** this course is taught as it is

Generative AI policy

- The instructor did not use generative AI during the preparation of the teaching material
- **Generative AI shall not be used in any part of the final project** (code and report)
 - Spell-checking and other generic writing assistants are allowed



Questions?

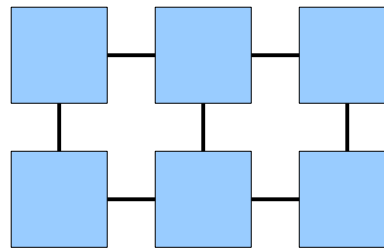
Intro to parallel programming

High Performance Computing

- Many applications need a lot of computing power
 - Weather forecast, climate modeling, physics simulation, product engineering, 3D animation, finance, ...
- Why?
 - To solve more complex problems
 - To solve the same problem in less time
 - To solve the same problem more accurately
 - To make better use of available computing resources

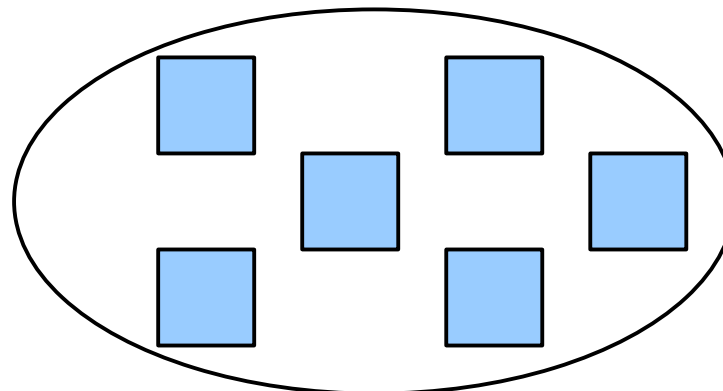
Parallel Computing (HPC) vs Cloud Computing (HTC)

- In **High Performance Computing** (HPC) we break applications into **strongly coupled** pieces executed on **specialized hardware** (CPUs and/or interconnection networks)



Interconnection network is important

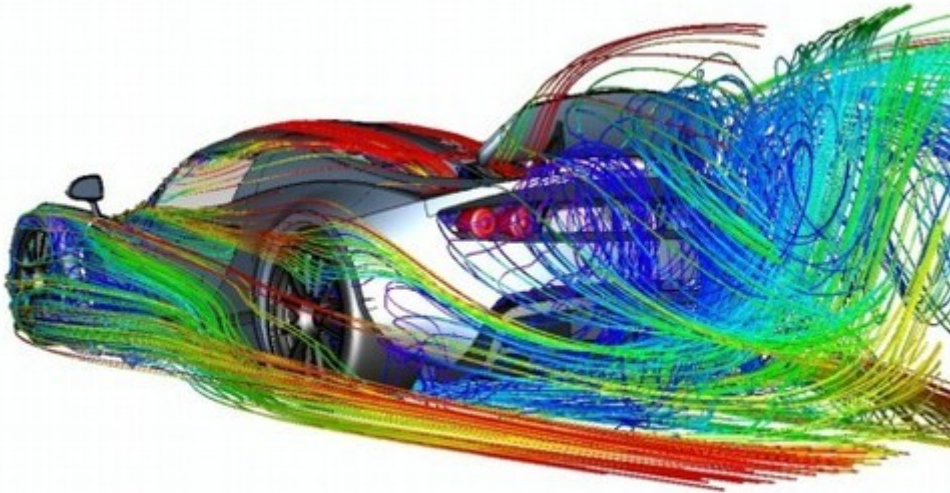
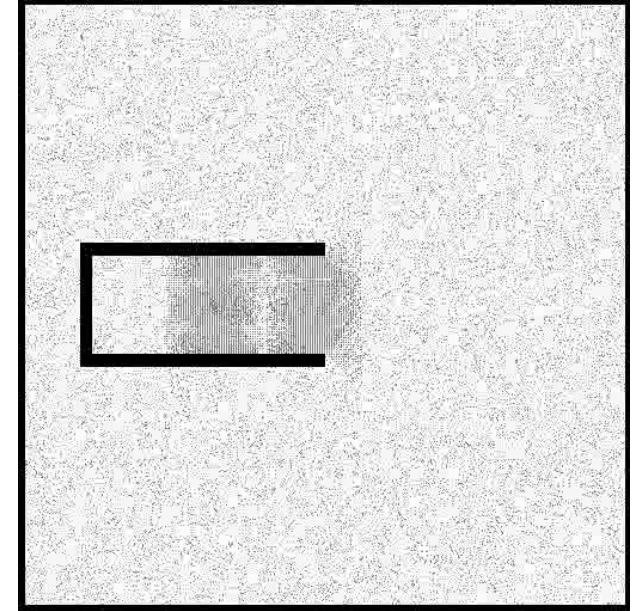
- In **High Throughput Computing** (HTC) we break applications into **loosely coupled** pieces executed on **commodity hardware**



Interconnection network is less important

High Performance Computing

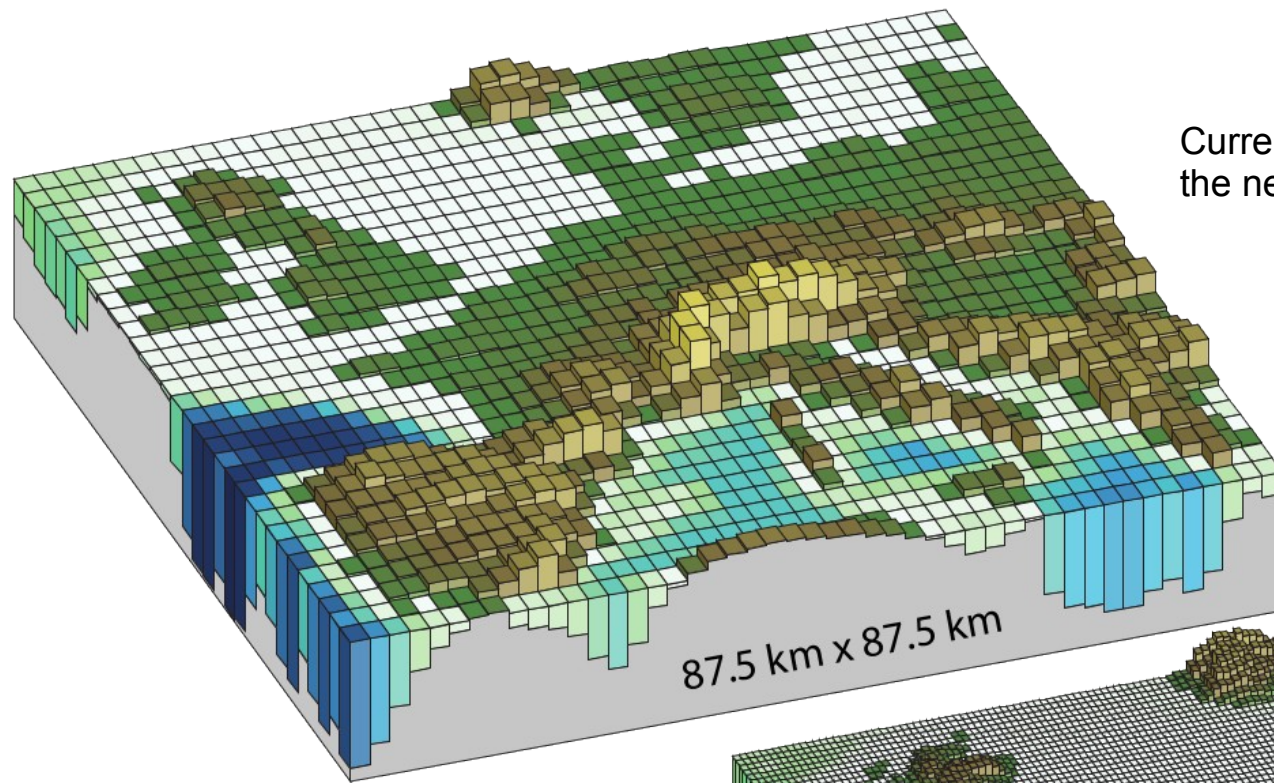
Numerical Wind Tunnel



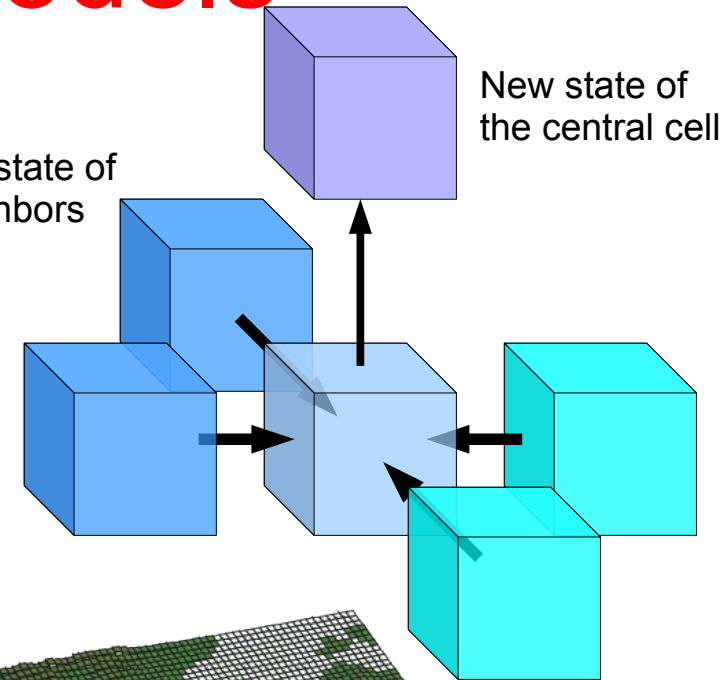
nce Computing

Source: <http://ecomodder.com/forum/showthread.php/random-wind-tunnel-smoke-pictures-thread-26678-12.html>

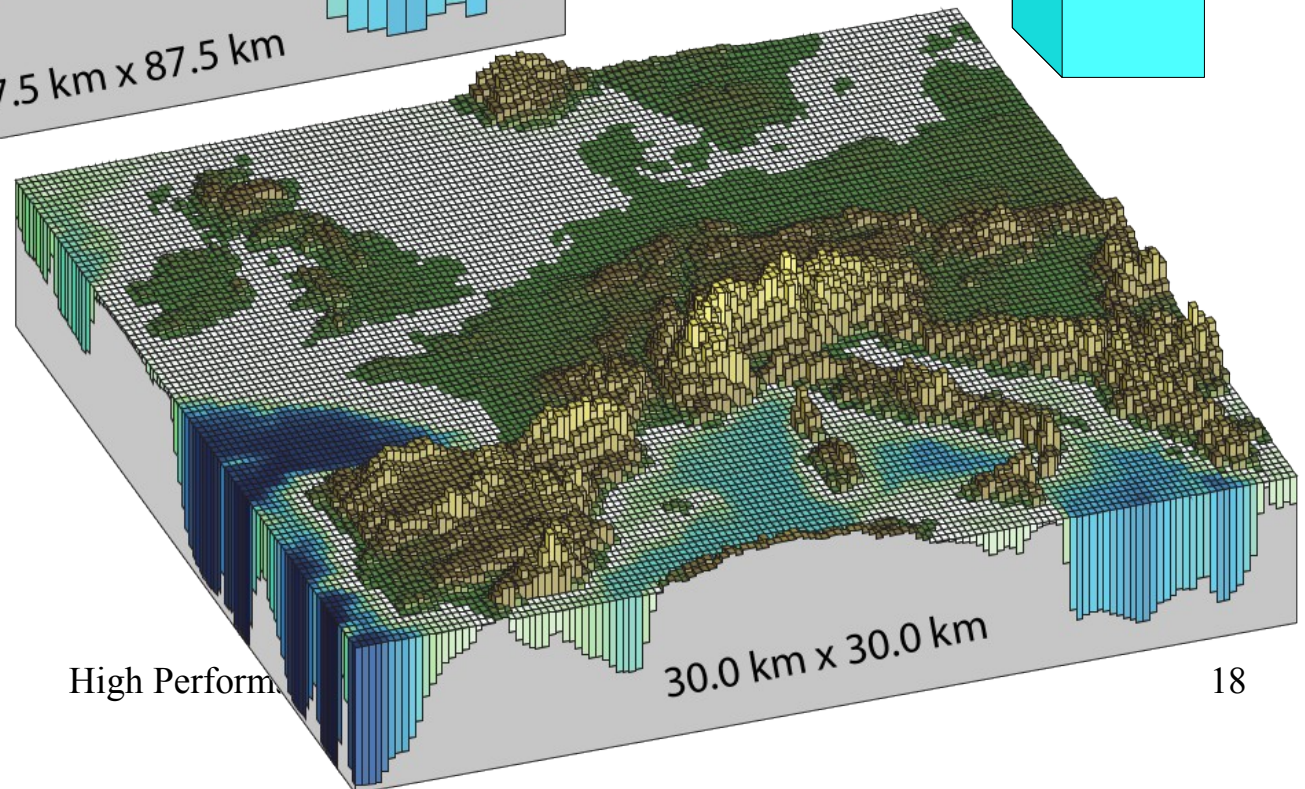
Global Climate models



Current state of the neighbors

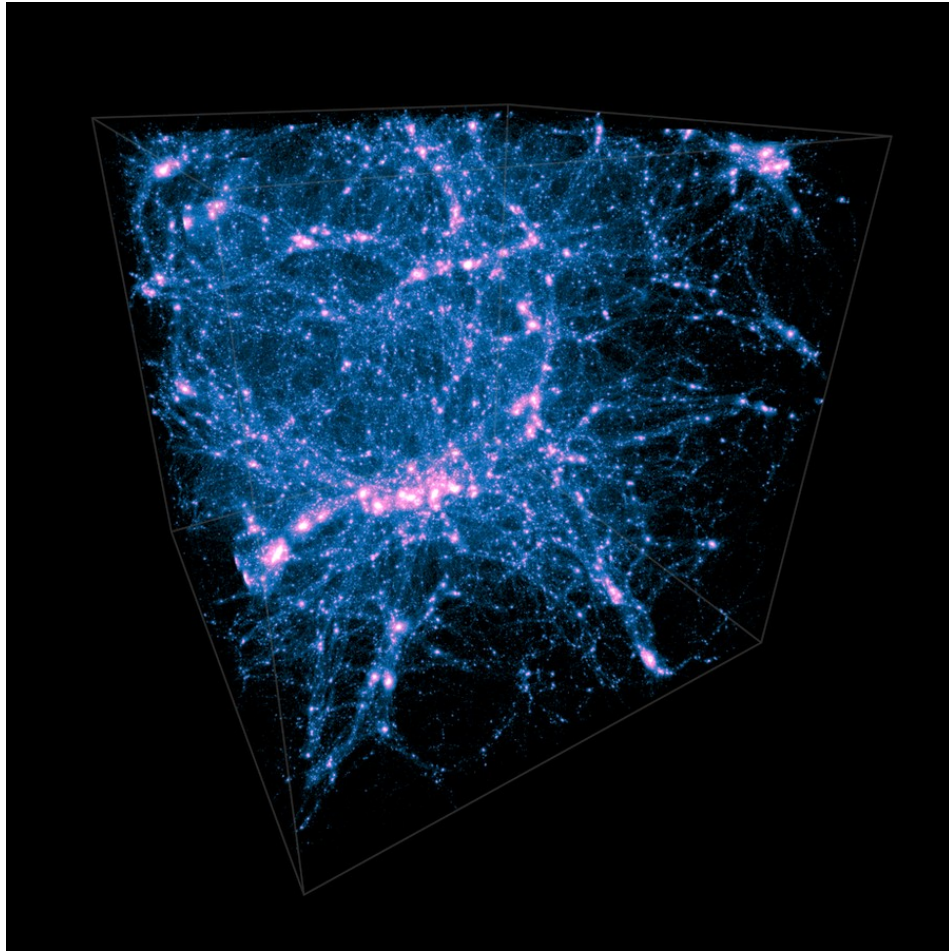


Source: Chapter 1 of *Climate Change 2013: The Physical Science Basis*
<https://www.ipcc.ch/report/ar5/wg1/>



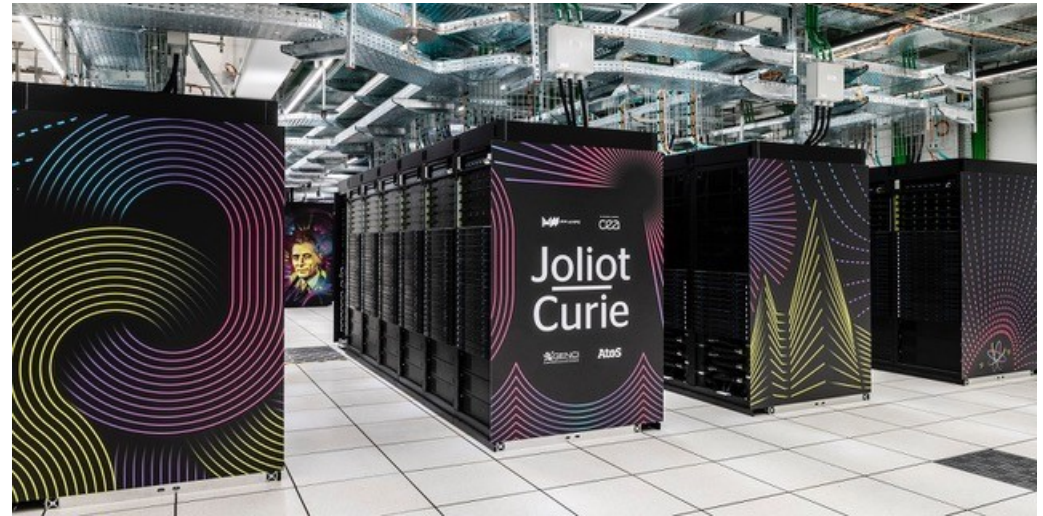
High Perform

Cosmological simulation: the Illustris project



“The Illustris simulations were run on supercomputers in France, Germany, and the US. The largest was run on 8,192 compute cores, and took 19 million CPU hours”

https://www.illustris-project.org/movies/illustris_movie_rot_sub_frame.mp4



Most of the Illustris simulation was executed in the Curie supercomputer with 440,000 x86 cores in nearly 5,000 nodes, 22 petaflop/s peak computing power

```
for each particle p {  
    compute the force on p by all other  
    particles p';  
    compute the new position of p using  
    Newton's second law;  
}
```

Moore's Law

"The number of transistors on an IC doubles every 24 months"

- That **used** to mean that every new generation of processors was based on smaller transistors

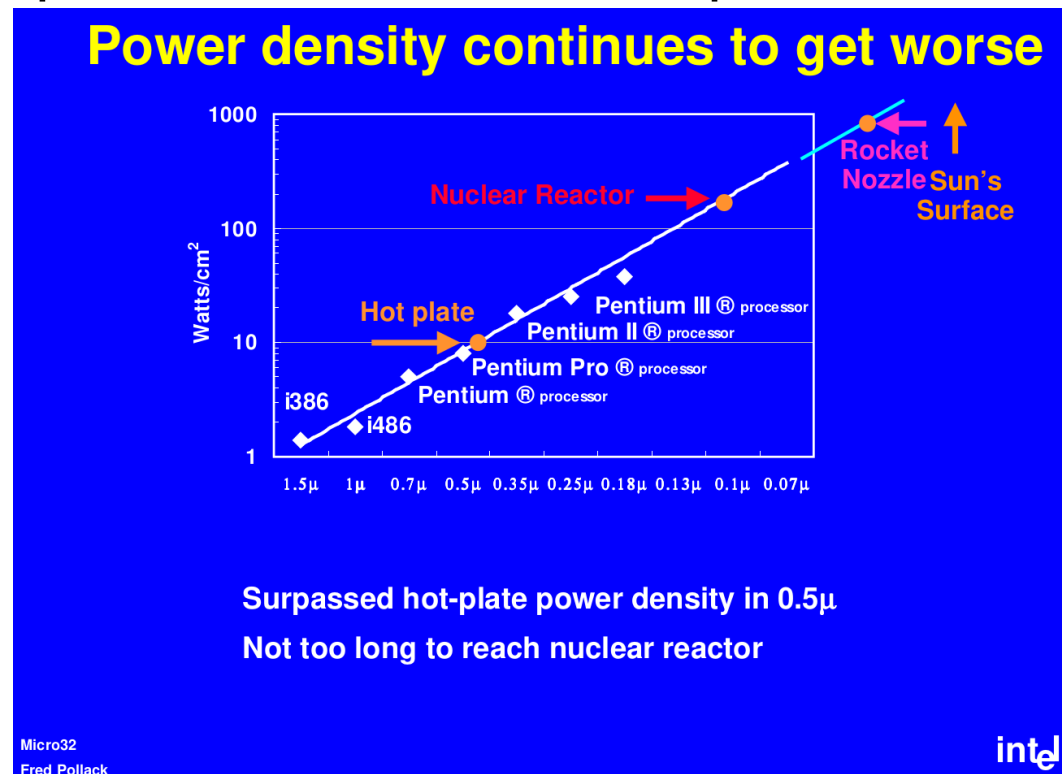
Moore, G.E., *Cramming more components onto integrated circuits*. Electronics, 38(8), April 1965



Gordon E.
Moore
(1929–)

Physics lesson

- Smaller transistors → Faster processor
- Faster processor → Higher power consumption
- Higher power consumption → More heat produced
- More heat produced → Unreliable processor



Power

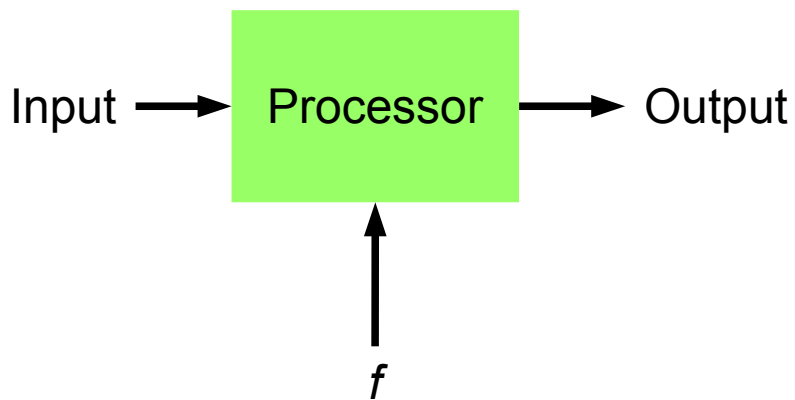
- The power required by an IC (e.g., a processor) can be expressed as

$$Power = C \times V^2 \times f$$

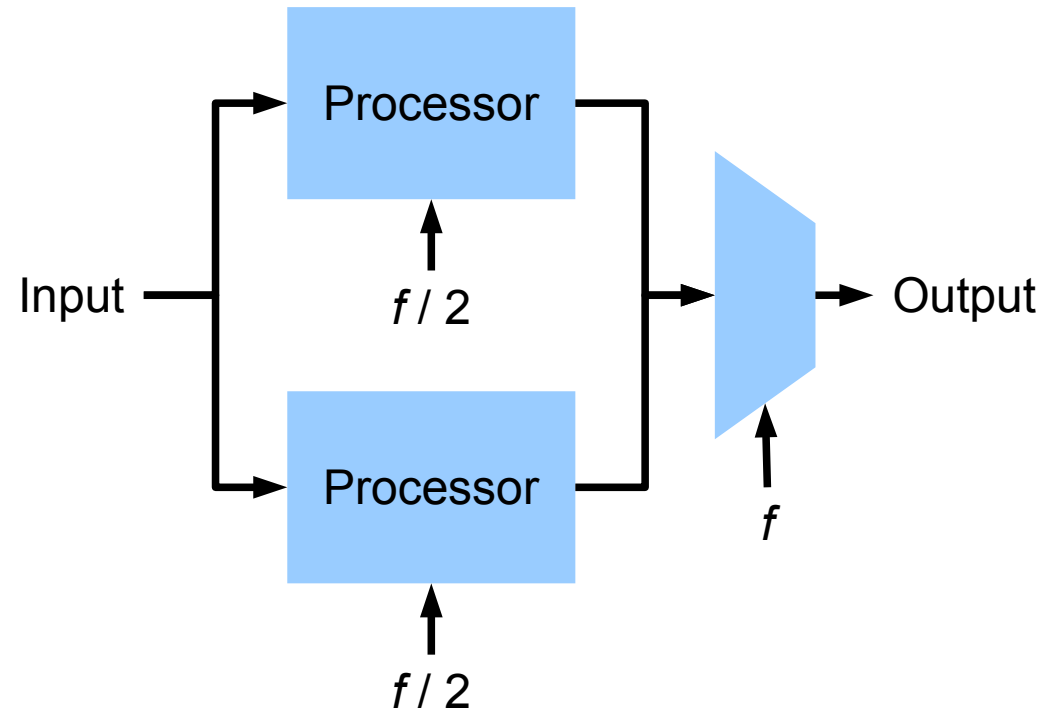
where:

- C is the *capacitance* (ability of a circuit to store energy)
- V is the *voltage*
- f is the *frequency* at which the processor operates

Power

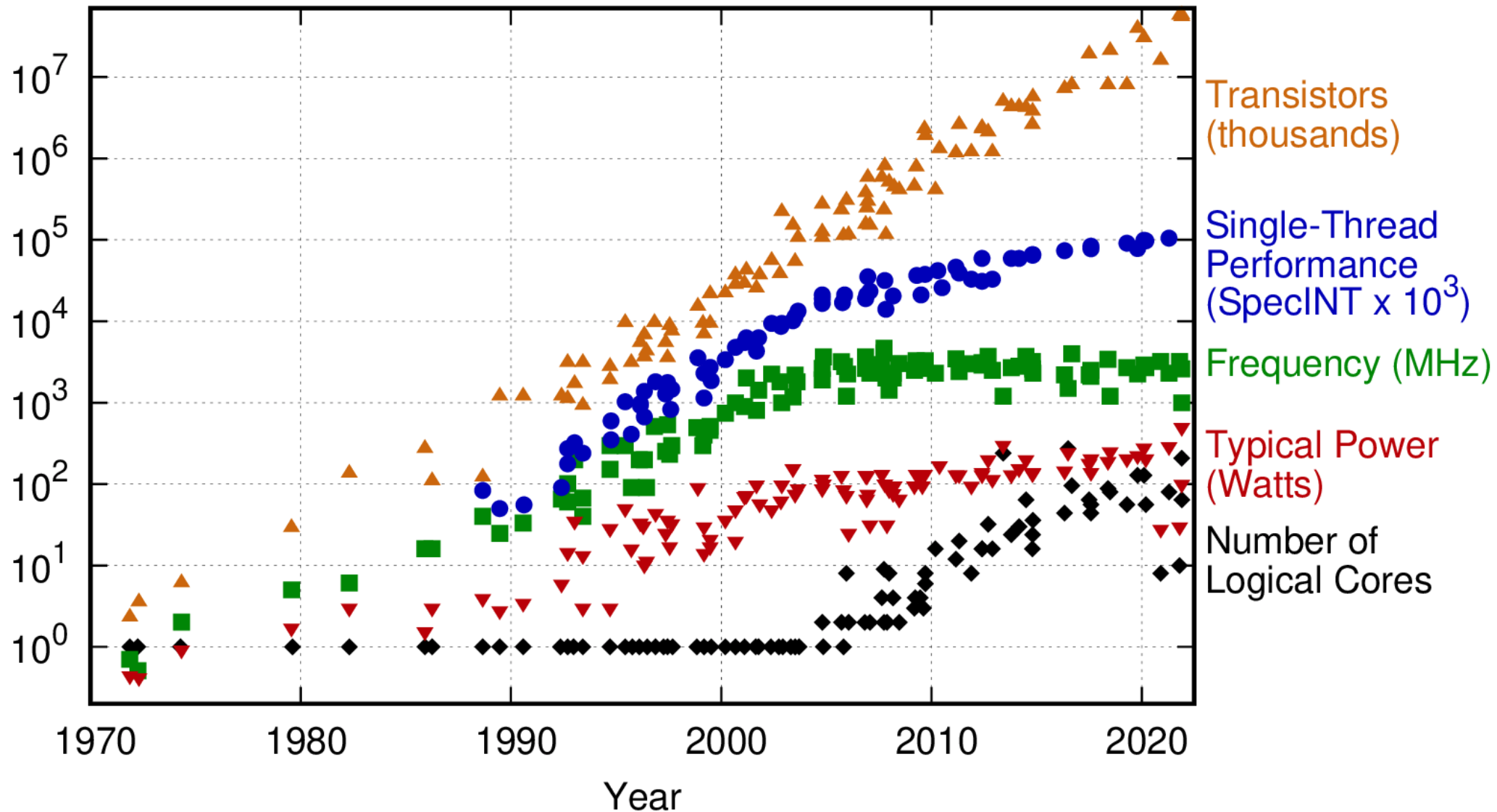


Capacitance C
Voltage V
Frequency f
Power = $C V^2 f$



Capacitance $2.2 C$
Voltage $0.6 V$
Frequency $0.5 f$
Power = $0.396 C V^2 f$

50 Years of Microprocessor Trend Data

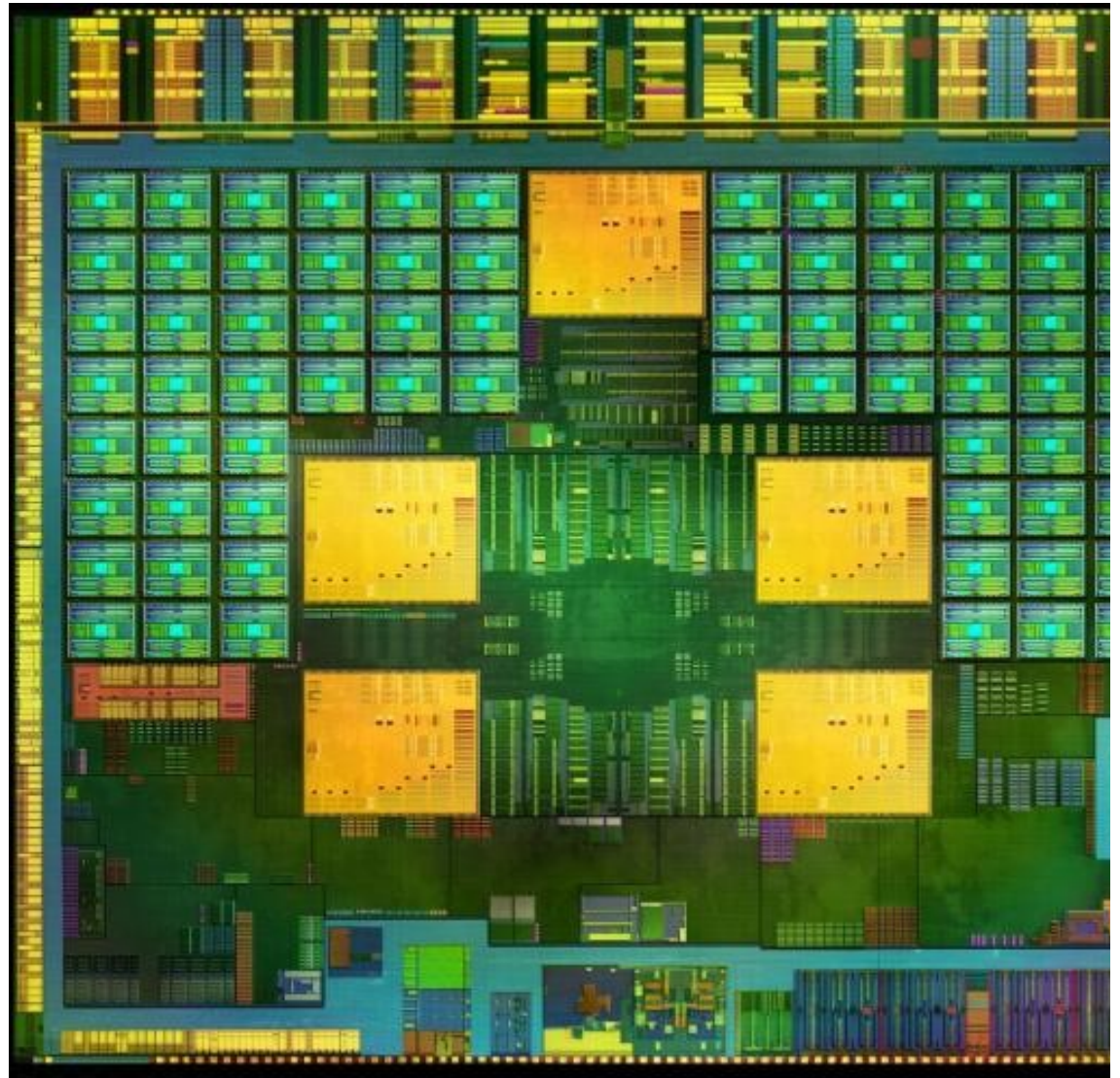


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

Source: <https://github.com/karlrupp/microprocessor-trend-data>

What happens today?

- HW designers create processors with more cores
- Result:
 - parallel hardware is **ubiquitous**
 - parallel software is **rare**
- The challenge
 - Make parallel software as common as parallel hardware



High Performance Computing

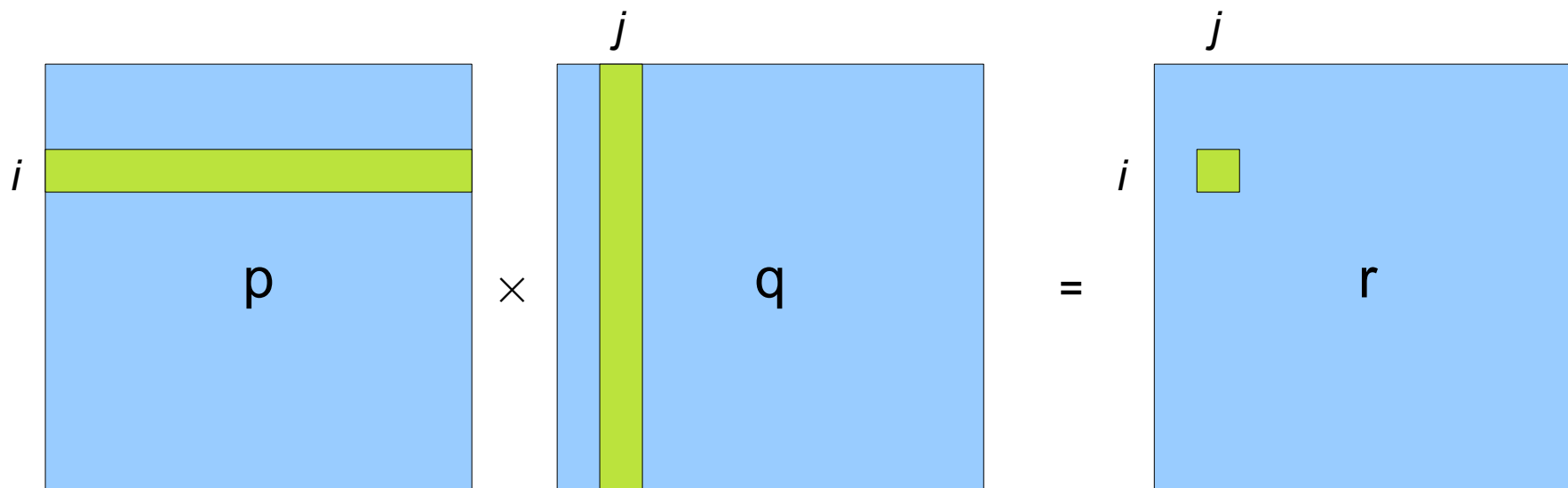
NVIDIA Tegra 4 SoC

Parallel programming in brief

- **Decompose** the problem in sub-problems
- **Distribute** sub-problems to the available execution units
- **Solve** sub-problems
 - **Cooperate** to solve sub-problems, if necessary
- **Combine** the partial results
- **Goals**
 - Reduce the wall-clock time
 - Balance the workload across execution units
 - Reduce communication and synchronization overhead

A case study in dense matrix-matrix multiplication

Matmul



$$r_{ij} = \sum_{k=0}^{n-1} p_{ik} q_{kj}$$

Environment

Feature	Specification
Processor type	Intel(R) Core(TM) i7-9800X
Clock frequency	3.80 GHz
Processor chips	1
Processing cores	8 (physical)
Hyperthreading	2 way (→ 16 “virtual” cores)
Floating-point unit	742 Gflops
L1-icache	32 KB × 8
L1-dcache	32 KB × 8
L2-cache	1 MB × 8
L3-cache	16 MB (shared)
DRAM	32 GB
Python	3.10.12
gcc	11.4.0, <code>-std=c99 -Wall -Wpedantic -fopenmp -mavx2 -mfma -O2</code>
Java	11.0.24

Version 1: Python

```
# matmul-test.py
import sys, random

n = 4096

p = [[random.random() for row in range(n)]
      for col in range(n)]
q = [[random.random() for row in range(n)]
      for col in range(n)]
r = [[0 for row in range(n)]
      for col in range(n)]

start = time()
for i in range(n):
    for j in range(n):
        for k in range(n):
            r[i][j] += p[i][k] * q[k][j]
elapsed = time() - start
print(elapsed)
```

Running time
= **19617** seconds
= **5.4** hours

$2n^3$ floating-point
operations in
19617s
≈ 7 MFLOPS

Peak **≈ 742**
GFLOPS

Python gets
0.0009% of peak

Version 2: Java

```
import java.util.Random;
import java.io.*;
public class matmul_test {
    final static int n = 4096;
    static double[][] p = new double[n][n];
    static double[][] q = new double[n][n];
    static double[][] r = new double[n][n];
    public static void main(String[] args) {
        Random rnd = new Random();
        for (int i=0; i<n; i++) {
            for (int j=0; j<n; j++) {
                p[i][j] = rnd.nextDouble();
                q[i][j] = rnd.nextDouble();
                r[i][j] = 0;
            }
        }
        final long start = System.nanoTime();
        for (int i=0; i<n; i++)
            for (int j=0; j<n; j++)
                for (int k=0; k<n; k++)
                    r[i][j] += p[i][k] * q[k][j];
        final long stop = System.nanoTime();
        System.out.println((stop - start)*1e-9);
    }
}
```

Running time
= **838** seconds
= **13,9** minutes

23 times faster
than Python

Version 3: C

```
void matmul(const float* p,
            const float* q,
            float *r, int n)
{
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            r[i*n + j] = 0;

    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            for (int k=0; k<n; k++)
                r[i*n + j] += p[i*n + k] * q[k*n + j];
}
```

Running time
= **595** seconds
= **9,9** minutes

33 times faster
than Python

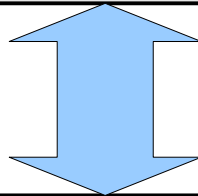
Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06

Reordering loops

- In this particular examples, the three nested loops can be permuted and still produce the same result

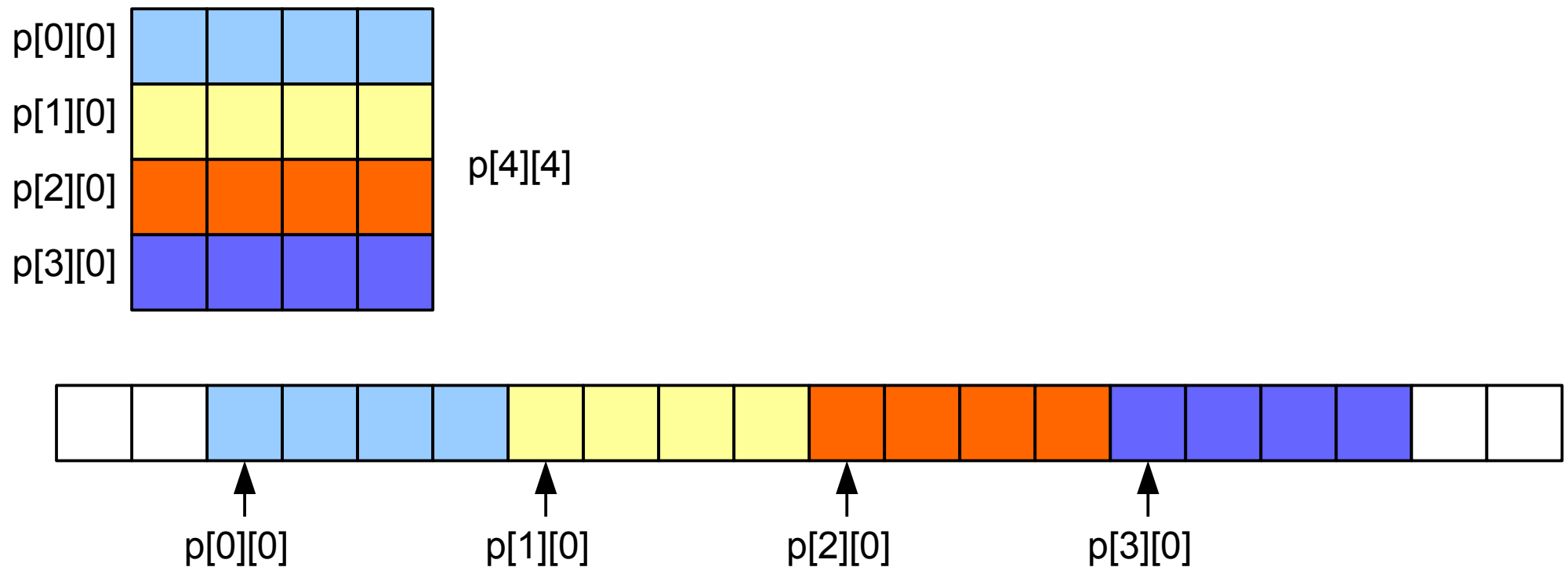
```
for (int i=0; i<n; i++)  
  for (int j=0; j<n; j++)  
    for (int k=0; k<n; k++)  
      r[i*n+j] += p[i*n+k] * q[k*n+j];
```



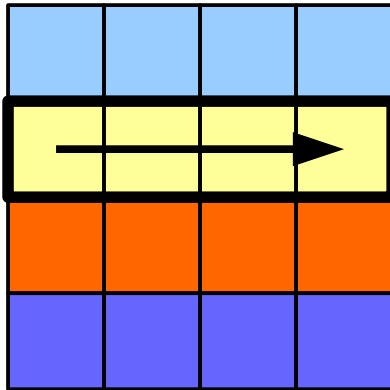
```
for (int k=0; k<n; k++)  
  for (int i=0; i<n; i++)  
    for (int j=0; j<n; j++)  
      r[i*n+j] += p[i*n+k] * q[k*n+j];
```

Matrix representation

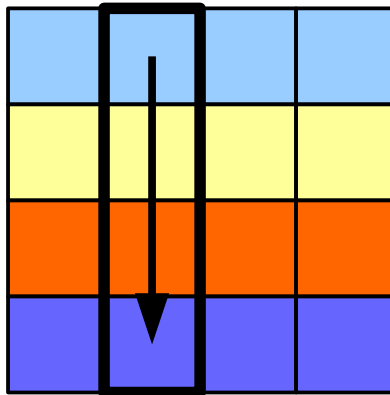
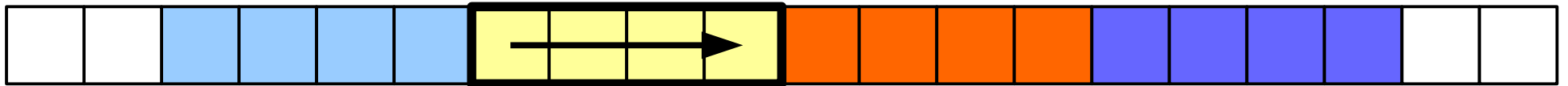
- Matrices in C are stored in *row-major order*
 - Elements of each row are contiguous in memory
 - Adjacent rows are contiguous in memory



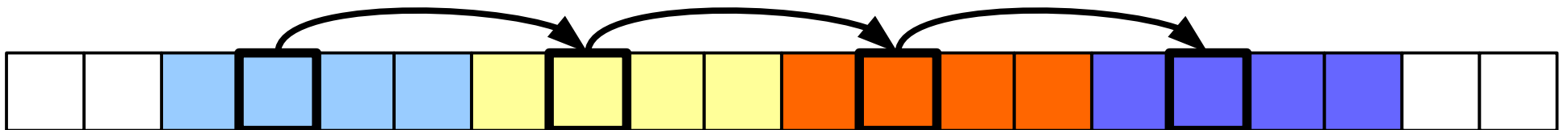
Row-wise vs column-wise access



Row-wise access is OK: the data is contiguous in memory, so the cache helps (spatial locality)

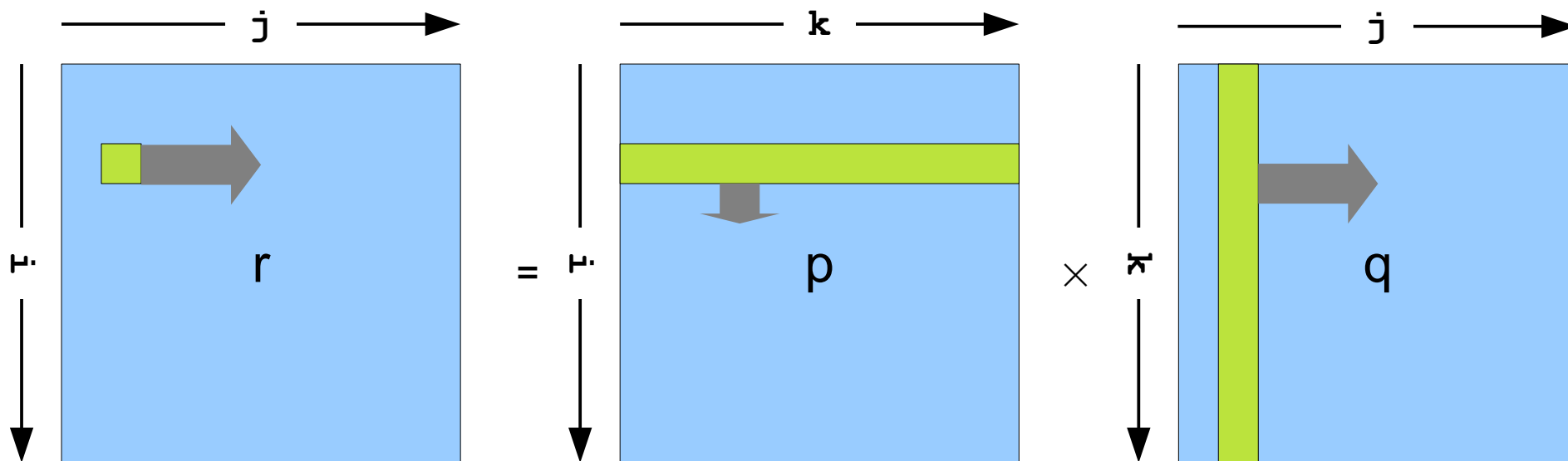


Column-wise access is **NOT OK**: the accessed elements are not contiguous in memory (*strided* access) so the cache does NOT help



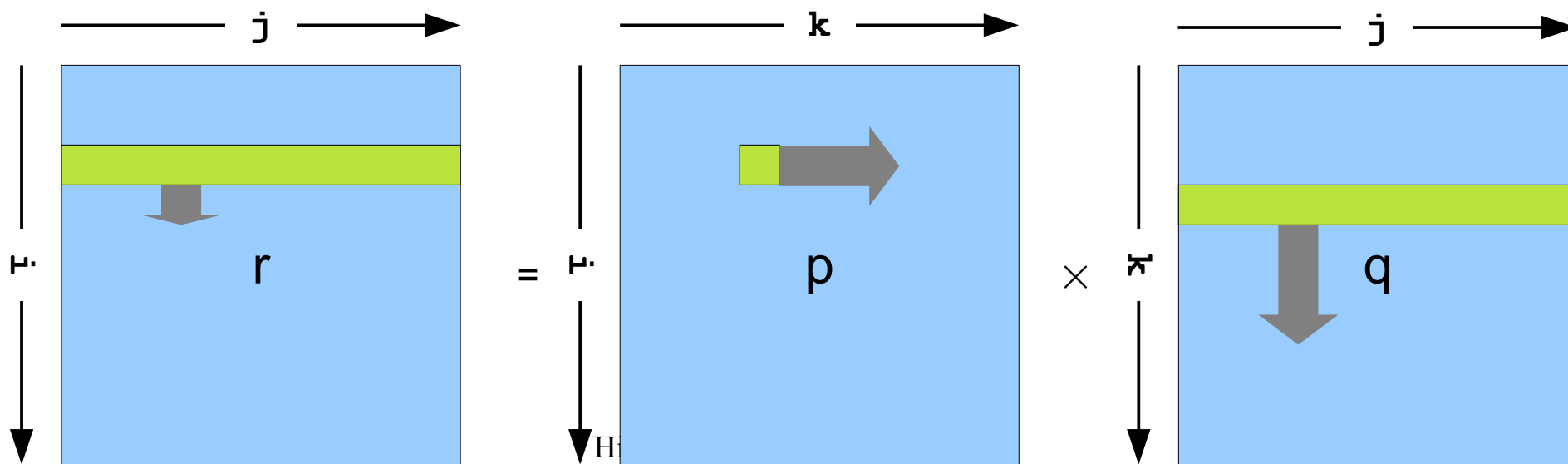
595 s

i, j, k



32 s

i, k, j



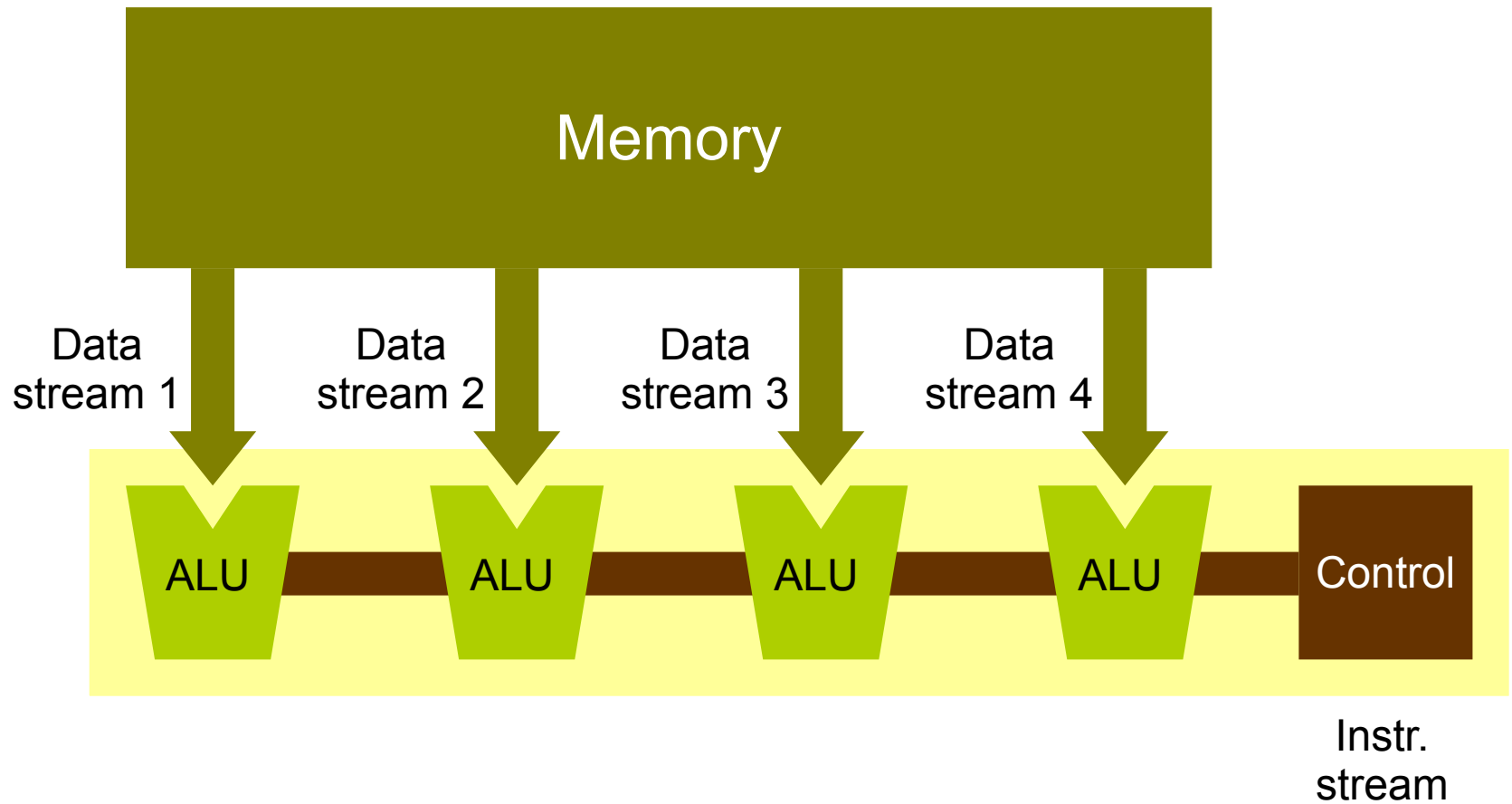
Reordering loops

Permutation	Time (s)	GFLOPS
i, j, k	595,85	0,23
i, k, j	32,06	4,29
j, i, k	343,08	0,40
j, k, i	824,92	0,17
k, i, j	36,65	3,75
k, j, i	1097,55	0,13

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06
+ Loop reordering	32.06	4.29	0.58%	611.93

SIMD instructions



```

void matmul_simd_ikj( const float *p, const float *q,
                      float *r, int n)
{
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j += VLEN) {
            __m256 *ptr = (__m256*)&r[i*n + j];
            *ptr = _mm256_setzero_ps();
        }
    }

    for (int i=0; i<n; i++) {
        for (int k=0; k<n; k++) {
            const __m256 pv = _mm256_broadcast_ss(&p[i*n + k]);
            for (int j=0; j<n; j += VLEN) {
                __m256 rv = _mm256_load_ps( &r[i*n + j] );
                const __m256 qv = _mm256_load_ps( &q[k*n + j] );
                rv = _mm256_fmadd_ps(pv, qv, rv);
                _mm256_store_ps( &r[i*n + j], rv );
            }
        }
    }
}

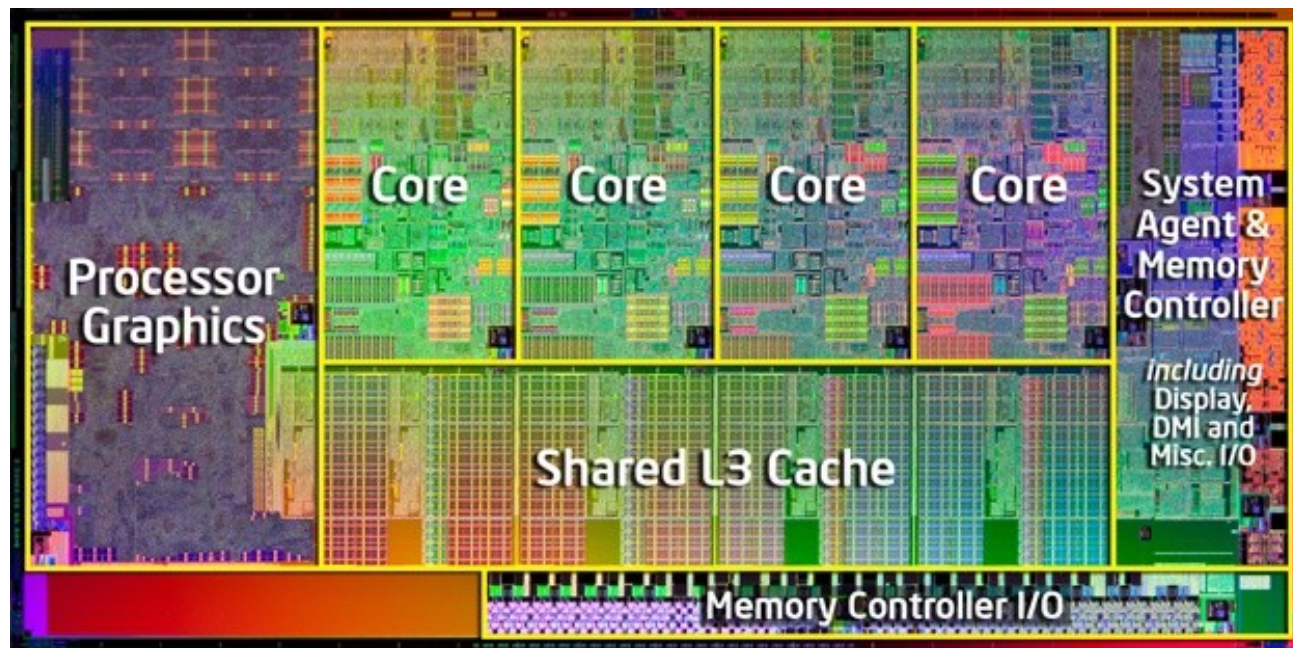
```

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06
+ Loop reordering	32.06	4.29	0.58%	611.93
+ SIMD	19.37	7.09	0.96%	1012.66

Multicore parallelism

- Modern processors have multiple independent execution units (“cores”)
 - This machine has 8 of them, each one seen by the OS as two “virtual” cores (Intel *HyperThreading*)
 - Virtual cores share some physical resources and are less powerful than “true” physical cores



```

void matmul_simd_ikj( const float *p, const float *q,
                      float *r, int n) {
#pragma omp parallel default(none) shared(p,q,r,n)
{
#pragma omp for
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j += VLEN) {
            __m256 *ptr = (__m256*)&r[i*n + j];
            *ptr = _mm256_setzero_ps();
        }
    }
#pragma omp for
    for (int i=0; i<n; i++) {
        for (int k=0; k<n; k++) {
            const __m256 pv = _mm256_broadcast_ss(&p[i*n + k]);
            for (int j=0; j<n; j += VLEN) {
                __m256 rv = _mm256_load_ps( &r[i*n + j] );
                const __m256 qv = _mm256_load_ps( &q[k*n + j] );
                rv = _mm256_fmadd_ps(pv, qv, rv);
                _mm256_store_ps( &r[i*n + j], rv );
            }
        }
    }
}
}

```

Matmul 4096 × 4096

	Time (s)	GFLOPS	% of peak	Speedup
Python	19617.28	0.01	0.00%	1.00
Java	838.81	0.16	0.02%	23.39
C	593.47	0.23	0.03%	33.06
+ Loop reordering	32.06	4.29	0.58%	611.93
+ SIMD	19.37	7.09	0.96%	1012.66
+ OpenMP	1.76	77.96	10.51%	11127.21